

Module 1: Build A Product Catalog Agent With Role-Based Access Control

This notebook builds the local prototype of the Product Catalog Agent.

You will learn how the agent discovers tools through MCP, how a role policy controls which tools are available to each persona, and how the local behavior contract is exported for later evaluation and deployment. The central idea is simple: the same agent can serve different users safely when tool access is filtered by role before the model acts.

Step 1: Load The Local Development Context

This cell imports dependencies, resolves paths, and loads the workshop state from setup.

Use the output to confirm that the notebook can see the product data, local MCP server, and configuration files. If this step fails, later agent calls usually fail for path or environment reasons rather than agent logic.

```
In [1]: import boto3
import json
import os
import sys
from pathlib import Path
from datetime import datetime

def _find_section_dir():
    start = Path.cwd().resolve()
    for parent in [start, *start.parents]:
        for candidate in (parent, parent / "01-single-agent-prototype"):
            if (candidate / "agents" / "product_catalog_agent.py").is_file() and (
                candidate / "mcp_servers" / "product_mcp_server.py"
            ).is_file():
                return candidate.resolve()
    raise FileNotFoundError(
        "Could not locate 01-single-agent-prototype. Open this notebook from the wo"
        "or the 01-single-agent-prototype folder."
    )

SECTION_DIR = _find_section_dir()
AGENTS_DIR = SECTION_DIR / "agents"
if str(AGENTS_DIR) not in sys.path:
    sys.path.insert(0, str(AGENTS_DIR))

print(f"Workshop section directory: {SECTION_DIR}")

# Get AWS region
```

```

session = boto3.Session()
REGION = session.region_name or 'us-west-2'
print(f"AWS Region: {REGION}")

# Set environment variables for tools
os.environ['AWS_REGION'] = REGION

# Get table names from SSM (set by workshop infrastructure)
ssm = boto3.client('ssm', region_name=REGION)

try:
    os.environ['PRODUCTS_TABLE_NAME'] = ssm.get_parameter(Name='ecommerce-workshop-
    print(f"Products Table: {os.environ['PRODUCTS_TABLE_NAME']}")
except Exception as e:
    print(f>Note: Could not retrieve SSM parameters ({e})")
    print("Using default table names - ensure infrastructure is ready")
    os.environ['PRODUCTS_TABLE_NAME'] = 'ecommerce-workshop-products'

```

Workshop section directory: /workshop/01-single-agent-prototype
 AWS Region: us-east-1
 Products Table: ecommerce-workshop-products

Step 2: Understand The MCP Tool Surface

This cell introduces the tools the agent can call through the local MCP server.

The key concept is separation of responsibilities: the model decides what it needs, while the MCP server owns the actual product catalog operations. Before you test the agent, inspect the tool names and descriptions so you know what the agent is allowed to use.

```

In [2]: # Verify MCP server exists
mcp_server_path = SECTION_DIR / 'mcp_servers' / 'product_mcp_server.py'
print(f"Product MCP Server: {mcp_server_path} - {'exists' if mcp_server_path.exists

```

Product MCP Server: /workshop/01-single-agent-prototype/mcp_servers/product_mcp_server.py - exists

Step 3: Validate Tool Discovery Against The Policy Contract

This cell compares the live MCP tool inventory with the versioned tool catalog and role policy.

You should learn how the workshop prevents drift between documented tools and executable tools. A clean result means the catalog, `config/tool_policy.json`, and server agree on tool names, descriptions, and role access before any agent is created.

```

In [3]: from strands import Agent
from strands.models import BedrockModel
from strands.tools.mcp import MCPClient
from mcp import StdioServerParameters

```

```

from mcp.client.stdio import stdio_client
from config_loader import load_agent_behavior_config

# Model configuration now comes from the Local behavior contract.
BEHAVIOR_CONFIG = load_agent_behavior_config(SECTION_DIR)
MODEL_CONFIG = BEHAVIOR_CONFIG.model_config
SONNET_MODEL_ID = MODEL_CONFIG["model_id"]
print(f"Model: {SONNET_MODEL_ID}")
print(f"Prompt version: {BEHAVIOR_CONFIG.prompt_version}")
print(f"Tool policy version: {BEHAVIOR_CONFIG.tool_policy_version}")

# Connect to the product MCP server
server_params = StdioServerParameters(
    command=sys.executable,
    args=[str(mcp_server_path)],
    env={
        **os.environ,
        "AWS_REGION": REGION,
        "PRODUCTS_TABLE": os.environ.get('PRODUCTS_TABLE_NAME', 'ecommerce-workshop')
    }
)

mcp_client = MCPClient(lambda: stdio_client(server_params))
mcp_client.__enter__()

# Discover all tools from the MCP server
all_tools = mcp_client.list_tools_sync()
print(f"\nDiscovered {len(all_tools)} tools from MCP server:")
for tool in all_tools:
    print(f"  - {tool.tool_name}")

```

Model: global.anthropic.claude-sonnet-4-6

Prompt version: product-catalog-prompts-v1

Tool policy version: product-catalog-tool-policy-v1

Discovered 11 tools from MCP server:

- search_products
- get_product_details
- check_inventory
- get_product_recommendations
- compare_products
- get_return_policy
- create_product
- update_product
- delete_product
- update_inventory
- update_pricing

```

In [4]: # Validate discovered MCP tools against the Local behavior contract.
discovered_tool_names = [tool.tool_name for tool in all_tools]
catalog_tool_names = set(BEHAVIOR_CONFIG.tool_catalog_by_name())
policy_tool_names = {
    tool_name
    for role in BEHAVIOR_CONFIG.tool_policy["roles"]
    for tool_name in BEHAVIOR_CONFIG.tools_for_role(role)
}

```

```

customer_policy_tools = BEHAVIOR_CONFIG.tools_for_role("customer")
admin_policy_tools = BEHAVIOR_CONFIG.tools_for_role("admin")
admin_only_policy_tools = [
    tool_name for tool_name in admin_policy_tools if tool_name not in customer_policy_tools
]

missing_from_catalog = sorted(set(discovered_tool_names) - catalog_tool_names)
missing_from_server = sorted(policy_tool_names - set(discovered_tool_names))
customer_admin_overlap = sorted(set(customer_policy_tools) & set(admin_only_policy_tools))

print("Policy/catalog validation:")
print(f"  Discovered MCP tools: {len(discovered_tool_names)}")
print(f"  Catalog tools: {len(catalog_tool_names)}")
print(f"  Policy tools: {len(policy_tool_names)}")
print(f"  Customer policy tools: {len(customer_policy_tools)}")
print(f"  Admin-only policy tools: {len(admin_only_policy_tools)}")
print(f"  Missing from catalog: {missing_from_catalog or 'none'}")
print(f"  Missing from MCP server: {missing_from_server or 'none'}")

assert not missing_from_catalog, "Every discovered MCP tool should have catalog metadata"
assert not missing_from_server, "Every policy tool should exist on the MCP server"
assert not customer_admin_overlap, "Customer tools should not include admin-only tools"

mcp_client.__exit__(None, None, None)
print("MCP discovery client cleaned up; role-filtered agents will open their own MCP connections.")

```

Policy/catalog validation:

```

  Discovered MCP tools: 11
  Catalog tools: 11
  Policy tools: 11
  Customer policy tools: 6
  Admin-only policy tools: 5
  Missing from catalog: none
  Missing from MCP server: none

```

MCP discovery client cleaned up; role-filtered agents will open their own MCP connections.

Why Tool Availability Comes Before Prompting

The agent prompt matters, but tool availability is the stronger control.

This section shows that role-based access is enforced by the tools made available to the agent, not only by instructions in the prompt. Keep this in mind when reading the customer and admin tests: the model can only call tools included in its role-filtered tool list.

Step 4: Create Role-Filtered Agents

This cell creates agent instances for different user roles.

The learning goal is to see how the same product catalog capabilities become different experiences for customers and administrators. Customer agents receive read-only tools; admin agents receive read and write tools. The output should make that tool split visible.

```
In [5]: # Add agents to path and import the RBAC-enabled agent
%load_ext autoreload
%autoreload 2

from product_catalog_agent import (
    BEHAVIOR_CONFIG,
    ProductCatalogAgent,
    UserSession,
    create_product_catalog_agent,
    CUSTOMER_TOOLS,
    ADMIN_TOOLS,
    ADMIN_ONLY_TOOLS,
    CUSTOMER_PERSONAS,
    ADMIN_PERSONAS,
)

print("RBAC Configuration (loaded from config files):")
print(f"  Agent: {BEHAVIOR_CONFIG.agent_name} ({BEHAVIOR_CONFIG.agent_version})")
print(f"  Prompt version: {BEHAVIOR_CONFIG.prompt_version}")
print(f"  Tool policy version: {BEHAVIOR_CONFIG.tool_policy_version}")
print(f"  Tool catalog version: {BEHAVIOR_CONFIG.tool_catalog_version}")
print(f"  Customer tools ({len(CUSTOMER_TOOLS)}): {CUSTOMER_TOOLS}")
print(f"  Admin-only tools ({len(ADMIN_ONLY_TOOLS)}): {ADMIN_ONLY_TOOLS}")
print(f"  Total admin tools ({len(ADMIN_TOOLS)}): {len(ADMIN_TOOLS)}")
```

```
RBAC Configuration (loaded from config files):
  Agent: ProductCatalogAgent (local-rbac-v1)
  Prompt version: product-catalog-prompts-v1
  Tool policy version: product-catalog-tool-policy-v1
  Tool catalog version: product-catalog-tool-catalog-v1
  Customer tools (6): ['search_products', 'get_product_details', 'check_inventory',
'get_product_recommendations', 'compare_products', 'get_return_policy']
  Admin-only tools (5): ['create_product', 'update_product', 'delete_product', 'update_inventory', 'update_pricing']
  Total admin tools (11): 11
```

Inspect The Local Behavior Contract

This cell previews the versioned configuration that defines the local prototype.

The behavior contract records prompt version, tool catalog version, role policy version, model settings, and MCP server details. Later notebooks use these version markers to connect evaluation and deployment evidence back to the prototype you tested here.

```
In [6]: contract_preview = BEHAVIOR_CONFIG.base_manifest()
contract_preview["role_policy"] = {
    role: BEHAVIOR_CONFIG.tools_for_role(role)
    for role in BEHAVIOR_CONFIG.tool_policy["roles"]
}
print(json.dumps(contract_preview, indent=2))
```

```

{
  "schema_version": "1.0",
  "module": "01-single-agent-prototype",
  "execution_mode": "local",
  "local_only": true,
  "generated_at": "2026-09-20T06:29:29Z",
  "agent": {
    "name": "ProductCatalogAgent",
    "version": "local-rbac-v1",
    "description": "Local Strands prototype for the Product Catalog Agent with confi
g-driven role-based tool access."
  },
  "model": {
    "provider": "bedrock",
    "model_id": "global.anthropic.claude-sonnet-4-6",
    "region_source": "notebook_or_runtime",
    "temperature": 0.3,
    "max_tokens": 1500
  },
  "config": {
    "agent_config_path": "config/agent_config.json",
    "prompt_version": "product-catalog-prompts-v1",
    "prompt_paths": {
      "customer": "config/prompts/customer.md",
      "admin": "config/prompts/admin.md"
    },
    "tool_policy_path": "config/tool_policy.json",
    "tool_policy_version": "product-catalog-tool-policy-v1",
    "tool_catalog_path": "config/tool_catalog.json",
    "tool_catalog_version": "product-catalog-tool-catalog-v1"
  },
  "mcp_server": {
    "name": "product-service",
    "path": "mcp_servers/product_mcp_server.py",
    "transport": "stdio"
  },
  "role_policy": {
    "customer": [
      "search_products",
      "get_product_details",
      "check_inventory",
      "get_product_recommendations",
      "compare_products",
      "get_return_policy"
    ],
    "admin": [
      "search_products",
      "get_product_details",
      "check_inventory",
      "get_product_recommendations",
      "compare_products",
      "get_return_policy",
      "create_product",
      "update_product",
      "delete_product",
      "update_inventory",

```

```

        "update_pricing"
    ]
}
}

```

Step 5: Create User Personas

This cell creates local user sessions for the customer and admin personas.

The persona object is a lightweight stand-in for identity context. It gives each test a role, name, and user metadata so you can see how the same question changes when the caller has different permissions. After running it, confirm the displayed personas have the expected role, user ID, and metadata, because those fields drive the role-filtered tests that follow.

```

In [7]: # Define test personas
customer_session = UserSession(
    user_id="CUST-1001",
    role="customer",
    email="john.smith@email.com",
    name="John Smith"
)

admin_session = UserSession(
    user_id="ADMIN-001",
    role="admin",
    email="alice.admin@company.com",
    name="Alice (Admin)"
)

print("Test Personas:")
print(f"  Customer: {customer_session.name} ({customer_session.email}) - role: {customer_session.role}")
print(f"  Admin:      {admin_session.name} ({admin_session.email}) - role: {admin_session.role}")

```

Test Personas:

```

Customer: John Smith (john.smith@email.com) - role: customer
Admin:      Alice (Admin) (alice.admin@company.com) - role: admin

```

Step 6: Test The Customer Persona

This cell sends customer-style prompts to the customer agent.

Read the output in two ways: first, whether the response is useful for product discovery; second, whether the agent avoids admin-only actions. Successful customer behavior includes search, details, recommendations, comparison, inventory checks, and safe refusal for catalog mutation.

```

In [8]: # Create agent with CUSTOMER role
customer_agent = create_product_catalog_agent(
    region=REGION,
    user_session=customer_session
)

```

```

user_info = customer_agent.get_user_info()
print(f"Agent created for: {user_info['name']} (role: {user_info['role']})")
print(f"Available tools ({user_info['tools_available']}): {user_info['tools']}")
customer_manifest = customer_agent.get_agent_manifest()
print(f"Manifest role: {customer_manifest['role']['resolved']} | policy: {customer_

```

Agent created for: John Smith (role: customer)
 Available tools (6): ['search_products', 'get_product_details', 'check_inventory', 'get_product_recommendations', 'compare_products', 'get_return_policy']
 Manifest role: customer | policy: product-catalog-tool-policy-v1 | prompt: product-catalog-prompts-v1

```

In [9]: # Customer Test 1: Search (should work)
print("=" * 60)
print("Customer Test 1: Product Search (ALLOWED)")
print("=" * 60)
response = customer_agent("I'm looking for a good gaming keyboard with RGB lighting")
print(f"\nResponse: {response}")

```

INFO | strands.telemetry.metrics | Creating Strands MetricsClient

Customer Test 1: Product Search (ALLOWED)

Response: Here's what we have for you:

🎮 Gaming Mechanical Keyboard RGB – \$159.99
 ★ **4.6/5** | ✅ **In Stock** (60 units available)

****Key Features:****

- 🌈 ****Per-key RGB lighting**** with 16.8 million colors
- 🛠️ ****Hot-swappable switches**** (Cherry MX compatible) – swap switches without soldering!
- 💻 ****Full-size layout**** with dedicated media keys and volume wheel
- ⌨️ ****Full N-key rollover**** (anti-ghosting) – every keypress registers perfectly
- 🔌 ****USB-C detachable cable**** for easy transport
- 🖐️ ****Detachable wrist rest**** included

****Purchase Confidence:****

- 🛡️ ****2-year warranty****
- 🔄 ****30-day return policy****

This is a solid choice for gamers! The hot-swappable switches are a standout feature – you can customize the feel of your keyboard without any technical expertise. The full N-key rollover ensures no missed inputs during intense gaming sessions.

Would you like to:

- ****Compare**** this with other options?
- ****See recommendations**** for matching gaming accessories (mouse, headset, etc.)?
- ****Ask any questions**** about the keyboard?

```
In [10]: # Customer Test 2: Recommendations (should work)
print("=" * 60)
print("Customer Test 2: Recommendations (ALLOWED)")
print("=" * 60)
response = customer_agent("Can you recommend some audio products for a music lover?")
print(f"\nResponse: {response}")
```

=====
Customer Test 2: Recommendations (ALLOWED)
=====

Response: Here are some fantastic audio products perfect for a music lover like you!



🎧 1. Wireless Bluetooth Headphones – \$79.99

****Best for: Everyday listening at home or on the go****

- Over-ear design with ****memory foam ear cushions**** for long listening sessions
- ****Active noise cancellation**** to block out distractions
- Impressive ****40-hour battery life**** – nearly a week of daily use!
- ✅ In Stock

🎧 2. Noise Canceling Earbuds – \$149.99

****Best for: Commuting & workouts****

- True wireless with ****adaptive noise cancellation****
- ****Transparency mode**** to let in ambient sound when needed
- ****8-hour battery per charge**** (plus more from the case)
- ✅ Availability not confirmed – I can check if needed!

🗣️ 3. Portable Bluetooth Speaker – \$199.99

****Best for: Sharing music with friends & outdoor use****

- ****360-degree sound**** for immersive listening
- ****Waterproof**** – great for outdoor adventures
- ****20-hour battery life****
- ****Party mode**** to pair multiple speakers together
- ✅ In Stock

🏆 My Picks Based on Your Needs:

| Use Case | Recommended Product |

|---|---|

| 🏠 Home/Office listening | Wireless Bluetooth Headphones (\$79.99) |

| 🚶 On-the-go/commuting | Noise Canceling Earbuds (\$149.99) |

| 🎉 Social/outdoor use | Portable Bluetooth Speaker (\$199.99) |

Would you like me to:

- 📄 ****Get full details**** on any of these products?
- ⚖️ ****Compare**** the headphones and earbuds side by side?
- 📦 ****Check inventory**** for any specific product?

```
In [11]: # Customer Test 3: Attempt admin action (should be REFUSED)
print("=" * 60)
print("Customer Test 3: Create Product (SHOULD BE REFUSED)")
print("=" * 60)
```

```
response = customer_agent("Create a new product called 'Super Speaker' with price $  
print(f"\nResponse: {response}")
```

```
=====
```

Customer Test 3: Create Product (SHOULD BE REFUSED)

```
=====
```

Response: Sorry that I couldn't do so!

Creating, updating, or deleting products is not something I'm able to help with. My role is limited to helping you **find, explore, and learn about existing products** in our catalog.

If you need to add or manage products, please reach out to our **store administrators or product management team** directly.

In the meantime, here's what I **can** help you with:

- 🔍 **Search** for existing audio products
- 📋 **View details** on current products
- ⚖️ **Compare** products side by side
- 📦 **Check inventory** availability

Is there anything else I can assist you with, John?

Interpreting The Customer Refusal

This is a negative test: the correct behavior is refusal.

A good refusal should identify that the user requested an admin operation, avoid calling write tools, and redirect the user to what they can do. This pattern becomes important later when RBAC behavior is evaluated and traced.

```
In [12]: # Customer Test 4: Attempt to delete a product (should be REFUSED)  
print("=" * 60)  
print("Customer Test 4: Delete Product (SHOULD BE REFUSED)")  
print("=" * 60)  
response = customer_agent("Delete the product PROD-001 from the catalog")  
print(f"\nResponse: {response}")
```

=====

Customer Test 4: Delete Product (SHOULD BE REFUSED)

=====

Response: Sorry that I couldn't do so!

Deleting products from the catalog is outside of my capabilities. I can only help you **browse and explore** existing products – any product management tasks like deletions need to be handled by our **store administrators or product management team**.

Here's what I **can** help you with instead:

- 🔍 **Search** for products across our catalog
- 📋 **Get details** on any existing product
- ⚖️ **Compare** multiple products
- 📦 **Check inventory** availability
- 💡 **Get personalized recommendations**

Is there anything else I can help you with, John? 😊

```
In [13]: # Clean up customer agent
customer_agent.cleanup()
print("Customer agent cleaned up")
```

Customer agent cleaned up

Step 7: Test The Admin Persona

This cell sends administrator-style prompts to the admin agent.

The key thing to observe is controlled expansion of capability. The admin agent can use write tools, but it should still be specific, auditable, and careful when creating, updating, deleting, or changing inventory/pricing.

```
In [14]: # Create agent with ADMIN role
admin_agent = create_product_catalog_agent(
    region=REGION,
    user_session=admin_session
)

user_info = admin_agent.get_user_info()
print(f"Agent created for: {user_info['name']} (role: {user_info['role']})")
print(f"Available tools ({user_info['tools_available']}): {user_info['tools']}")
admin_manifest = admin_agent.get_agent_manifest()
print(f"Manifest role: {admin_manifest['role']['resolved']} | policy: {admin_manifest['policy']}")
```

Agent created for: Alice (Admin) (role: admin)
Available tools (11): ['search_products', 'get_product_details', 'check_inventory', 'get_product_recommendations', 'compare_products', 'get_return_policy', 'create_product', 'update_product', 'delete_product', 'update_inventory', 'update_pricing']
Manifest role: admin | policy: product-catalog-tool-policy-v1 | prompt: product-catalog-prompts-v1

```
In [15]: # Admin Test 1: Search still works (admin has all customer tools too)
print("=" * 60)
print("Admin Test 1: Product Search (ALLOWED)")
print("=" * 60)
# Search for all audio products with noise cancelling
response = admin_agent("Search for all audio products")
print(f"\nResponse: {response}")
```

```
=====
Admin Test 1: Product Search (ALLOWED)
=====
```

Response: Here's a summary of all **Audio** products found in the catalog:

#	Product ID	Name	Price	In Stock
1	PROD-001	Wireless Bluetooth Headphones	\$79.99	Yes
2	PROD-055	Noise Canceling Earbuds	\$149.99	Yes
3	PROD-033	Portable Bluetooth Speaker	\$199.99	Yes

Quick Descriptions:

- **PROD-001 - Wireless Bluetooth Headphones**: Premium over-ear headphones with 40-hour battery, active noise cancellation, and memory foam ear cushions.
- **PROD-055 - Noise Canceling Earbuds**: True wireless earbuds with adaptive noise cancellation, transparency mode, and 8-hour battery per charge.
- **PROD-033 - Portable Bluetooth Speaker**: Waterproof 360-degree sound speaker with 20-hour battery and multi-speaker party mode.

All 3 audio products are currently **in stock**. Would you like to view detailed specs, update any of these products, or perform any other catalog actions?

```
In [16]: # Admin Test 2: Create a new product
print("=" * 60)
print("Admin Test 2: Create Product (ALLOWED)")
print("=" * 60)
response = admin_agent(
    "Create a new product with ID PROD-200 called 'Premium Gaming Headset' "
    "in the Audio category for $129.99. Description: 'Professional gaming headset "
    "with 7.1 surround sound, detachable microphone, and RGB lighting.' "
    "Specifications: bluetooth 5.3, weight 320g, driver size 50mm. "
    "Set initial stock to 50 units."
)
print(f"\nResponse: {response}")
```

```
=====
Admin Test 2: Create Product (ALLOWED)
=====
```

Response:  **Product Successfully Created!**

Here's a summary of the new product added to the catalog:

```
| Field | Details |
|-----|-----|
| **Product ID** | PROD-200 |
| **Name** | Premium Gaming Headset |
| **Category** | Audio |
| **Price** | $129.99 |
| **Initial Stock** | 50 units |
| **Status** | Active |
```

📄 Full Details:

- **Description:** Professional gaming headset with 7.1 surround sound, detachable microphone, and RGB lighting.
- **Specifications:**
 - Bluetooth: 5.3
 - Weight: 320g
 - Driver Size: 50mm
- **Warranty:** 1 year manufacturer warranty **(default)**
- **Return Policy:** 30-day return policy **(default)**

The product is now live in the catalog and ready for purchase. Would you like to make any further changes, such as updating the warranty, adding a sale price, or anything else?

```
In [17]: # Verify product creation
from utils import get_product
import json

product = get_product(product_id='PROD-200')
print(json.dumps(product, indent=2))
```

```

{
  "success": true,
  "product_id": "PROD-200",
  "details": "Product: Premium Gaming Headset\nProduct ID: PROD-200\nCategory: Audio\nPrice: $129.99\n\nDescription:\nProfessional gaming headset with 7.1 surround sound, detachable microphone, and RGB lighting.\n\nSpecifications:\n  - bluetooth: 5.3\n  - weight: 320g\n  - driver_size: 50mm\n\nAvailability: In Stock\nStock Quantity: 50\n\nRating: 0 out of 5\nWarranty: 1 year manufacturer warranty\nReturn Policy: 30-day return policy\n",
  "product_data": {
    "rating": 0.0,
    "stock_quantity": 50.0,
    "created_date": "2026-09-20 06:33:45",
    "specifications": "{\n  \"bluetooth\": \"5.3\", \"weight\": \"320g\", \"driver_size\": \"50mm\"}",
    "in_stock": true,
    "product_id": "PROD-200",
    "name": "Premium Gaming Headset",
    "warranty": "1 year manufacturer warranty",
    "return_policy": "30-day return policy",
    "category": "Audio",
    "price": 129.99,
    "description": "Professional gaming headset with 7.1 surround sound, detachable microphone, and RGB lighting.",
    "updated_date": "2026-09-20 06:33:45"
  },
  "source": "product_catalog"
}

```

```

In [18]: # Admin Test 3: Update pricing with a sale
print("=" * 60)
print("Admin Test 3: Update Pricing (ALLOWED)")
print("=" * 60)
response = admin_agent(
    "Set a sale price of $99.99 for PROD-200 (regular price stays $129.99) "
    "until 2026-12-31"
)
print(f"\nResponse: {response}")

```

=====

Admin Test 3: Update Pricing (ALLOWED)

=====

Response:  **Sale Price Successfully Set for PROD-200!**

Field	Details
Product ID	PROD-200
Name	Premium Gaming Headset
Regular Price	\$129.99
Sale Price	\$99.99
Discount	23.1% off
Sale Ends	December 31, 2026

The regular price of **\$129.99** remains unchanged and will automatically be restored once the sale expires on **2026-12-31**. Would you like to make any other changes?

In [19]: *# Verify product pricing update*

```
from utils import get_product
import json
```

```
product = get_product(product_id='PROD-200')
print(json.dumps(product, indent=2))
```

```
{
  "success": true,
  "product_id": "PROD-200",
  "details": "Product: Premium Gaming Headset\nProduct ID: PROD-200\nCategory: Audio\nPrice: $129.99\nDescription:\nProfessional gaming headset with 7.1 surround sound, detachable microphone, and RGB lighting.\nSpecifications:\n - bluetooth: 5.3\n - weight: 320g\n - driver_size: 50mm\nAvailability: In Stock\nStock Quantity: 50\nRating: 0 out of 5\nWarranty: 1 year manufacturer warranty\nReturn Policy: 30-day return policy\n",
  "product_data": {
    "category": "Audio",
    "created_date": "2026-09-20 06:33:45",
    "description": "Professional gaming headset with 7.1 surround sound, detachable microphone, and RGB lighting.",
    "in_stock": true,
    "name": "Premium Gaming Headset",
    "price": 129.99,
    "product_id": "PROD-200",
    "rating": 0.0,
    "return_policy": "30-day return policy",
    "sale_end_date": "2026-12-31",
    "sale_price": 99.99,
    "specifications": "{\"bluetooth\": \"5.3\", \"weight\": \"320g\", \"driver_size\": \"50mm\"}",
    "stock_quantity": 50.0,
    "updated_date": "2026-09-20 06:34:16",
    "warranty": "1 year manufacturer warranty"
  },
  "source": "product_catalog"
}
```

```
In [20]: # Admin Test 4: Update inventory
print("=" * 60)
print("Admin Test 4: Update Inventory (ALLOWED)")
print("=" * 60)
response = admin_agent("Set the inventory for PROD-088 (4K webcam) to 100 units")
print(f"\nResponse: {response}")
```

```
=====
Admin Test 4: Update Inventory (ALLOWED)
=====
```

Response:  **Inventory Successfully Updated for PROD-088!**

Field	Details
Product ID	PROD-088
Name	Webcam 4K HDR
Previous Stock	0 units (Out of Stock)
New Stock	100 units
Status	 Now In Stock

The product was previously **out of stock** with a restock date of 2025-01-15, and is now back in stock with **100 units** available. Would you like to make any other changes?

```
In [21]: # Verify product inventory update
from utils import get_product
import json

product = get_product(product_id='PROD-088')
if product.get('success') and 'product_data' in product:
    print(f"PROD-088 inventory update verified: stock_quantity = {product['product_
else:
    print(f"Note: Could not verify PROD-088 update - {product.get('message', json.d
```

PROD-088 inventory update verified: stock_quantity = 100.0

```
In [22]: # Admin Test 5: Delete (discontinue) a product
print("=" * 60)
print("Admin Test 5: Delete Product (ALLOWED - soft delete)")
print("=" * 60)
# Explicit confirmation up-front: the admin prompt instructs the agent to
# "always confirm important changes", so without it the model sometimes
# asks for confirmation instead of deleting - and PROD-200 Leaks into
# later runs.
response = admin_agent(
    "Discontinue product PROD-200. Yes, I confirm the deletion."
)
print(f"\nResponse: {response}")
```

```
=====
Admin Test 5: Delete Product (ALLOWED - soft delete)
=====
```

Response:  ****Product Successfully Discontinued!****

Field	Details
Product ID	PROD-200
Name	Premium Gaming Headset
Action	Soft Delete (Discontinued)
Status	Removed from Active Catalog

 Important Notes:

- This was a ****soft delete**** – the product record is ****retained**** in the database for historical order and audit purposes.
- ****PROD-200** is no longer visible** to customers or searchable in the active catalog.
- The action has been ****timestamped**** and logged under your admin account (Alice).

If you ever need to reinstate this product, the record can be restored. Would you like to do anything else?

```
In [23]: # Clean up admin agent
admin_agent.cleanup()
print("Admin agent cleaned up")
```

Admin agent cleaned up

Step 8: Validate The Cross-Role Boundary

This cell tests behavior that crosses the customer/admin boundary.

The sequence creates or updates catalog state as an admin, then verifies what a customer can and cannot do with that state. This gives you a concrete trace of the RBAC lifecycle: admin mutation, customer read, customer denial for mutation, and admin verification.

```
In [24]: # Create both agents
admin_agent = create_product_catalog_agent(region=REGION, user_session=admin_session)
customer_agent = create_product_catalog_agent(region=REGION, user_session=customer_session)

print(f"Admin tools:    {admin_agent.get_user_info()['tools_available']} tools")
print(f"Customer tools: {customer_agent.get_user_info()['tools_available']} tools")
```

Admin tools: 11 tools
Customer tools: 6 tools

```
In [25]: # Step A: Admin creates a new product
print("=" * 60)
print("Step A: Admin creates product PROD-300")
print("=" * 60)
response = admin_agent(
    "Create product PROD-300 called 'Wireless Charging Pad' in Accessories category")
```

```

        "for $39.99. Description: 'Fast wireless charger compatible with all Qi devices
        "supports up to 15W charging.'" Specs: charging speed 15W, compatibility Qi, wei
        "Stock: 200 units."
    )
    print(f"\nAdmin Response: {response}")

```

=====
Step A: Admin creates product PROD-300
=====

Admin Response:  ****Product Successfully Created!****

Here's a summary of the new product added to the catalog:

Field	Details
Product ID	PROD-300
Name	Wireless Charging Pad
Category	Accessories
Price	\$39.99
Description	Fast wireless charger compatible with all Qi devices, supports up to 15W charging.
Charging Speed	15W
Compatibility	Qi
Weight	120g
Stock	200 units
Warranty	1 year manufacturer warranty *(default)*
Return Policy	30-day return policy *(default)*

****PROD-300**** is now live in the catalog under the ****Accessories**** category with ****200 units**** in stock. Let me know if you'd like to make any adjustments!

```

In [26]: # Step B: Customer can find the new product
print("=" * 60)
print("Step B: Customer searches for the new product")
print("=" * 60)
response = customer_agent("Do you have any wireless charging pads?")
print(f"\nCustomer Response: {response}")

```

```
=====
Step B: Customer searches for the new product
=====
```

Customer Response: Great news! We do carry a wireless charging pad. Here's what we have:

```
### 📱 Wireless Charging Pad – **$39.99** (PROD-300)
- **Category:** Accessories
- **Compatibility:** Works with all **Qi-enabled devices**
- **Charging Speed:** Supports up to **15W fast charging**
- **Availability:** ✅ In Stock
```

This is a solid, affordable option for fast wireless charging. Would you like to:

- 📄 **Get more details** on the charging pad (full specs, warranty, etc.)?
- 🔄 **Check the return policy** before purchasing?
- 🛒 Anything else I can help you with?

```
In [27]: # Step C: Customer tries to modify the product (SHOULD FAIL)
print("=" * 60)
print("Step C: Customer tries to change the price (SHOULD BE REFUSED)")
print("=" * 60)
response = customer_agent("Change the price of PROD-300 to $19.99")
print(f"\nCustomer Response: {response}")
```

```
=====
Step C: Customer tries to change the price (SHOULD BE REFUSED)
=====
```

Customer Response: Sorry that I couldn't do so! I'm only able to help customers find and learn about products – I don't have the ability to create, update, delete, or modify product details or pricing.

If you'd like to report a pricing concern or request a price change, please reach out to our support team directly. Is there anything else I can help you with? 😊

```
In [28]: # Step D: Clean up - admin deletes the test product
print("=" * 60)
print("Step D: Admin cleans up test product")
print("=" * 60)
# Same explicit confirmation as the PROD-200 delete above.
response = admin_agent(
    "Delete product PROD-300. Yes, I confirm the deletion."
)
print(f"\nAdmin Response: {response}")
```

```
=====
Step D: Admin cleans up test product
=====
```

Admin Response:  ****Product Successfully Deleted!****

Here's a summary of the action taken:

Field	Details
Product ID	PROD-300
Name	Wireless Charging Pad
Action	Soft Delete (Discontinued)
Status	Removed from active catalog

 ****Note:**** This was a ****soft delete**** – the product record for ****PROD-300**** has been marked as ***discontinued*** and is no longer visible in the active catalog. However, the record is ****retained in the system**** for order history and audit purposes. It cannot be purchased by customers going forward.

Let me know if you need anything else!

Cross-Role Boundary Summary

Use this summary to check whether the role boundary behaved as expected.

The table should show that read operations remain available to both roles where appropriate, while write operations remain admin-only. If a customer reaches an admin-only tool, the prototype contract is not safe enough for later deployment.

```
In [29]: # Clean up both agents
admin_agent.cleanup()
customer_agent.cleanup()
print("Both agents cleaned up")
```

Both agents cleaned up

Step 9: Export The Local Agent Contract

This cell saves the local behavior contract as a reusable artifact.

The export is not just documentation. It gives later sections a stable pointer to the prompt version, tool policy version, role behavior, and tested personas that produced the baseline behavior.

```
In [30]: section01_manifest_summary = {
        "customer": customer_agent.get_agent_manifest(),
        "admin": admin_agent.get_agent_manifest(),
    }

    for persona, manifest in section01_manifest_summary.items():
```

```
print("=" * 60)
print(f"{persona.title()} manifest")
print("=" * 60)
print(json.dumps({
    "agent": manifest["agent"],
    "model": manifest["model"],
    "config": manifest["config"],
    "role": manifest["role"],
    "available_tools": manifest["available_tools"],
    "tool_metadata": manifest["tool_metadata"],
}, indent=2))
```

```
=====
Customer manifest
=====
```

```
{
  "agent": {
    "name": "ProductCatalogAgent",
    "version": "local-rbac-v1",
    "description": "Local Strands prototype for the Product Catalog Agent with confi
g-driven role-based tool access."
  },
  "model": {
    "provider": "bedrock",
    "model_id": "global.anthropic.claude-sonnet-4-6",
    "region_source": "notebook_or_runtime",
    "temperature": 0.3,
    "max_tokens": 1500,
    "region": "us-east-1"
  },
  "config": {
    "agent_config_path": "config/agent_config.json",
    "prompt_version": "product-catalog-prompts-v1",
    "prompt_paths": {
      "customer": "config/prompts/customer.md",
      "admin": "config/prompts/admin.md"
    },
    "tool_policy_path": "config/tool_policy.json",
    "tool_policy_version": "product-catalog-tool-policy-v1",
    "tool_catalog_path": "config/tool_catalog.json",
    "tool_catalog_version": "product-catalog-tool-catalog-v1"
  },
  "role": {
    "requested": "customer",
    "resolved": "customer",
    "default_role_applied": false
  },
  "available_tools": [
    "search_products",
    "get_product_details",
    "check_inventory",
    "get_product_recommendations",
    "compare_products",
    "get_return_policy"
  ],
  "tool_metadata": [
    {
      "name": "search_products",
      "operation_type": "read",
      "risk_level": "low",
      "mutates_catalog": false,
      "requires_confirmation": false,
      "input_summary": "query, optional category, optional max_results",
      "allowed_roles": [
        "customer",
        "admin"
      ],
      "description": "Search the product catalog by query and optional category."
    }
  ]
}
```

```
},
{
  "name": "get_product_details",
  "operation_type": "read",
  "risk_level": "low",
  "mutates_catalog": false,
  "requires_confirmation": false,
  "input_summary": "product_id",
  "allowed_roles": [
    "customer",
    "admin"
  ],
  "description": "Retrieve detailed product information by product ID."
},
{
  "name": "check_inventory",
  "operation_type": "read",
  "risk_level": "low",
  "mutates_catalog": false,
  "requires_confirmation": false,
  "input_summary": "product_id",
  "allowed_roles": [
    "customer",
    "admin"
  ],
  "description": "Check stock quantity and availability for a product."
},
{
  "name": "get_product_recommendations",
  "operation_type": "read",
  "risk_level": "low",
  "mutates_catalog": false,
  "requires_confirmation": false,
  "input_summary": "context and optional max_recommendations",
  "allowed_roles": [
    "customer",
    "admin"
  ],
  "description": "Recommend products based on customer context."
},
{
  "name": "compare_products",
  "operation_type": "read",
  "risk_level": "low",
  "mutates_catalog": false,
  "requires_confirmation": false,
  "input_summary": "comma-separated product_ids",
  "allowed_roles": [
    "customer",
    "admin"
  ],
  "description": "Compare multiple products side by side."
},
{
  "name": "get_return_policy",
  "operation_type": "read",
```

```

        "risk_level": "low",
        "mutates_catalog": false,
        "requires_confirmation": false,
        "input_summary": "optional product_id",
        "allowed_roles": [
            "customer",
            "admin"
        ],
        "description": "Retrieve product-specific or store-wide return policy information."
    }
]
}

```

=====
Admin manifest
=====

```

{
  "agent": {
    "name": "ProductCatalogAgent",
    "version": "local-rbac-v1",
    "description": "Local Strands prototype for the Product Catalog Agent with config-driven role-based tool access."
  },
  "model": {
    "provider": "bedrock",
    "model_id": "global.anthropic.claude-sonnet-4-6",
    "region_source": "notebook_or_runtime",
    "temperature": 0.3,
    "max_tokens": 1500,
    "region": "us-east-1"
  },
  "config": {
    "agent_config_path": "config/agent_config.json",
    "prompt_version": "product-catalog-prompts-v1",
    "prompt_paths": {
      "customer": "config/prompts/customer.md",
      "admin": "config/prompts/admin.md"
    },
    "tool_policy_path": "config/tool_policy.json",
    "tool_policy_version": "product-catalog-tool-policy-v1",
    "tool_catalog_path": "config/tool_catalog.json",
    "tool_catalog_version": "product-catalog-tool-catalog-v1"
  },
  "role": {
    "requested": "admin",
    "resolved": "admin",
    "default_role_applied": false
  },
  "available_tools": [
    "search_products",
    "get_product_details",
    "check_inventory",
    "get_product_recommendations",
    "compare_products",
    "get_return_policy",
    "create_product",

```

```

    "update_product",
    "delete_product",
    "update_inventory",
    "update_pricing"
  ],
  "tool_metadata": [
    {
      "name": "search_products",
      "operation_type": "read",
      "risk_level": "low",
      "mutates_catalog": false,
      "requires_confirmation": false,
      "input_summary": "query, optional category, optional max_results",
      "allowed_roles": [
        "customer",
        "admin"
      ],
      "description": "Search the product catalog by query and optional category."
    },
    {
      "name": "get_product_details",
      "operation_type": "read",
      "risk_level": "low",
      "mutates_catalog": false,
      "requires_confirmation": false,
      "input_summary": "product_id",
      "allowed_roles": [
        "customer",
        "admin"
      ],
      "description": "Retrieve detailed product information by product ID."
    },
    {
      "name": "check_inventory",
      "operation_type": "read",
      "risk_level": "low",
      "mutates_catalog": false,
      "requires_confirmation": false,
      "input_summary": "product_id",
      "allowed_roles": [
        "customer",
        "admin"
      ],
      "description": "Check stock quantity and availability for a product."
    },
    {
      "name": "get_product_recommendations",
      "operation_type": "read",
      "risk_level": "low",
      "mutates_catalog": false,
      "requires_confirmation": false,
      "input_summary": "context and optional max_recommendations",
      "allowed_roles": [
        "customer",
        "admin"
      ],
    },
  ]
}

```

```

    "description": "Recommend products based on customer context."
  },
  {
    "name": "compare_products",
    "operation_type": "read",
    "risk_level": "low",
    "mutates_catalog": false,
    "requires_confirmation": false,
    "input_summary": "comma-separated product_ids",
    "allowed_roles": [
      "customer",
      "admin"
    ],
    "description": "Compare multiple products side by side."
  },
  {
    "name": "get_return_policy",
    "operation_type": "read",
    "risk_level": "low",
    "mutates_catalog": false,
    "requires_confirmation": false,
    "input_summary": "optional product_id",
    "allowed_roles": [
      "customer",
      "admin"
    ],
    "description": "Retrieve product-specific or store-wide return policy information."
  },
  {
    "name": "create_product",
    "operation_type": "write",
    "risk_level": "high",
    "mutates_catalog": true,
    "requires_confirmation": true,
    "input_summary": "product_id, name, category, price, description, specifications, stock_quantity",
    "allowed_roles": [
      "admin"
    ],
    "description": "Create a new product in the catalog."
  },
  {
    "name": "update_product",
    "operation_type": "write",
    "risk_level": "high",
    "mutates_catalog": true,
    "requires_confirmation": true,
    "input_summary": "product_id and JSON updates",
    "allowed_roles": [
      "admin"
    ],
    "description": "Modify product attributes such as name, description, specs, or category."
  },
  {

```

```

        "name": "delete_product",
        "operation_type": "write",
        "risk_level": "high",
        "mutates_catalog": true,
        "requires_confirmation": true,
        "input_summary": "product_id",
        "allowed_roles": [
            "admin"
        ],
        "description": "Soft delete a product by marking it discontinued."
    },
    {
        "name": "update_inventory",
        "operation_type": "write",
        "risk_level": "medium",
        "mutates_catalog": true,
        "requires_confirmation": true,
        "input_summary": "product_id, new_quantity, optional restock_date",
        "allowed_roles": [
            "admin"
        ],
        "description": "Adjust stock quantities and optional restock dates."
    },
    {
        "name": "update_pricing",
        "operation_type": "write",
        "risk_level": "high",
        "mutates_catalog": true,
        "requires_confirmation": true,
        "input_summary": "product_id, new_price, optional sale_price, optional sale_end_date",
        "allowed_roles": [
            "admin"
        ],
        "description": "Change regular pricing and optional sale pricing."
    }
]
}

```

Module 1 Summary

You built and tested the local Product Catalog Agent.

The important outcomes are: tool discovery works, customer/admin roles receive different tool sets, the customer role refuses catalog mutations, the admin role can perform controlled mutations, and the behavior contract is exported for evaluation and deployment work.

```

In [31]: # Save region for use in next module
%store REGION
print("Session data saved for Module 2!")

```

```
Stored 'REGION' (str)  
Session data saved for Module 2!
```

In []: